isn’t an overload for configuring the `MvcOptions`. Instead, you need to use the `AddMvcOptions()` extension method to configure the filters, as shown in the following listing.

**Listing 21.7 Applying filters globally to a Razor Pages application**

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.RazorPages()                                    ❶
    .AddMvcOptions(options =>                                    ❷
    {
        options.Filters.Add(new LogResourceFilter());            ❸
        options.Filters.Add(typeof(LogResourceFilter));          ❸
        options.Filters.Add<LogResourceFilter>();                ❸
    });
```

❶ This method doesn’t let you pass a lambda to configure the MvcOptions.

❷ You must use an extension method to add the filters to the MvcOptions object.

❸ You can configure the filters in any of the ways shown previously.

With potentially three different scopes in play, you’ll often find action methods that have multiple filters applied to them, some applied directly to the action method and others inherited from the controller or globally. Then the question becomes which filter runs first.

## *21.6 Understanding the order of filter execution*

You’ve seen that the filter pipeline contains five stages, one for each type of filter. These stages always run in the fixed order I described in sections 21.1 and 21.2. But within each stage, you can also have multiple filters of the same type (for example, multiple resource filters) that are part of a single